Building Your Agent

The details behind the Quick Start: how you define an agent, choose how it's served, write its instructions, and pick a model.

Everything about your agent lives in one declarative object — the `AgentDefinition`. You set the fields; the SDK turns it into a running, secured service.

The agent definition

```python
from pathlib import Path
import os

from ai_foundry_platform_sdk.agent import (
    AgentDefinition, AuthConfig, MemoryConfig, Skill, SkillPackage,
)
from ai_foundry_platform_sdk.model import ModelConfig

agent = AgentDefinition(
    name="my-agent",                    # Required
    description="What this agent does",    # Required
    version="1.0.0",

    # Instructions (see "Prompts" below)
    domain_prompt_id="pp-my-agent-v1",     # managed prompt ID, for production
    domain_prompt="You are a helpful ...", # inline fallback, for local dev

    # Model (optional — sensible defaults apply)
    model_config=ModelConfig(temperature=0.2, max_tokens=4096),

    # Capabilities
    tools=[...],                        # what the agent can do — see Tools & Files
    skills=[Skill(id="main", name="Main", description="Primary capability")],

    # Optional
```

```
    memory=MemoryConfig(enabled=True),    # remember conversations — see Memory

    # Who can use it (required — see Security & Access)
    auth=AuthConfig(allowed_groups=["<group-id-1>", "<group-id-2>"]),
)
```

Field	Required	What it's for
`name`	Yes	The agent's name.
`description`	Yes	A short description of what it does.
`domain_prompt_id` / `domain_prompt`	One of them	Your agent's instructions (see Prompts).
`tools`	No	External capabilities — run code, read files, query a DB. See Tools & Files.
`skills`	Yes (≥1)	Described capabilities, surfaced so others know what the agent does.
`memory`	No	Conversation persistence. See Memory.
`auth`	Yes	Which company groups may use the agent. See Security & Access.
`model_config`	No	Model overrides (temperature, max tokens, ...). Defaults work out of the box.
`hooks`	No	Custom logic at points in the request lifecycle (see Hooks).

A2A vs AG-UI

There are two ways to serve an agent. You pick one when you scaffold (`--protocol agui` or `--protocol a2a`) and it's reflected by which app function you call.

	AG-UI	A2A
Stands for	Agent-to-UI	Agent-to-Agent
Use it for	Agents people **chat with** through a frontend	**Backend** agents called by other agents or services
How it streams	Live streaming to a UI (server-sent events)	Request/response over the A2A protocol
Port (default)	8080	9000
App function	`create_agui_app(agent)`	`create_a2a_app(agent)`

```
# Chat agent (frontend-facing)
from ai_foundry_platform_sdk.app import create_agui_app
app = create_agui_app(agent)

# Backend agent (called by other agents/services)
from ai_foundry_platform_sdk.app import create_a2a_app
app = create_a2a_app(agent)
```

There's also `build_agent(...)` for **sub-agents** that run in-process inside another agent (for multi-step workflows) — no server, no protocol. See Multi-agent workflows.

Health checks

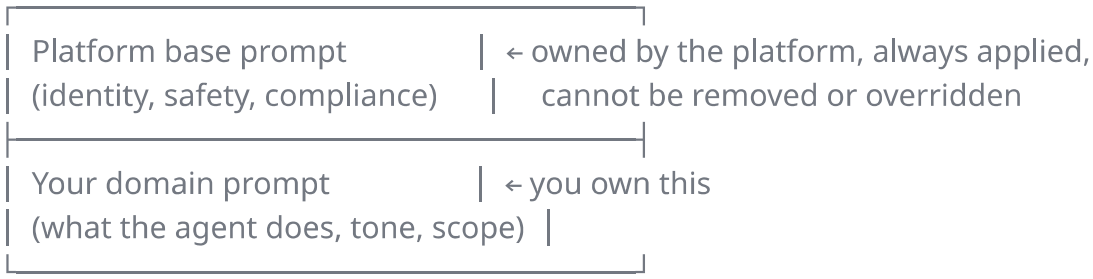
Every served agent automatically exposes two health endpoints used by the platform:

- `GET /ping` — liveness ("is the process alive?"), always returns 200.
- `GET /ready` — readiness ("can it serve requests?"), checks that the model key resolves and the gateway client is reachable.

You don't write these — they're wired in for you.

Prompts

An agent's behaviour is shaped by two layers of instructions:



The **base prompt** is always prepended and enforces safety and compliance. You can't switch it off. Your **domain prompt** sits on top and describes your agent.

Where your domain prompt comes from

The SDK resolves it in this order:

1. **`domain_prompt_id`** — loaded from the central prompt service (Portkey). Use this in production so prompt teams can iterate without redeploying.
2. **`domain_prompt`** — an inline string. Use this for local development.
3. **Neither** — only the base prompt applies.

A common pattern is to set both: the ID (from an environment variable) for production, and the inline string as a local fallback.

```
agent = AgentDefinition(
    domain_prompt_id=os.environ.get("MY_PROMPT_ID"),   # set in production
    domain_prompt="You are a helpful assistant for the R&D team.",  # local fallback
    ...
)
```

Don't fight the base prompt

At startup the SDK scans your domain prompt for phrases that try to override the platform's instructions (e.g. *"ignore previous instructions"*, *"you are now..."*) and warns you. Write your domain prompt as additive context — don't try to redefine the agent's identity or safety rules.

Models

All AI model calls go through the central **Portkey gateway** — you never handle a provider API key in code, and the platform can switch or fall back between models centrally. With no configuration, the agent uses the platform default model (currently **Gemini 3.5 Flash**).

```
from ai_foundry_platform_sdk.model import ModelConfig

agent = AgentDefinition(
    model_config=ModelConfig(
        model_id="@<virtual-key>/global.anthropic.claude-opus-4-8",
        temperature=0.0,     # optional
        max_tokens=8192,     # optional
    ),
    ...
)
```

Setting	Meaning
`model_id`	The gateway model slug to use. Omit for the platform default.
`temperature`	Sampling temperature. **Emitted only when you set it** — otherwise the model's own default applies.
`max_tokens`	Maximum response length. **Emitted only when you set it** — otherwise the model's own default applies.

Setting	Meaning
`top_p`, `stop_sequences`, `extra_params`	Advanced sampling controls.

> **Behaviour change in 1.3.** The SDK no longer injects a default `temperature` (previously `0.2`) or `max_tokens` (previously `4096`) when you leave them unset — the agent now inherits each model's own defaults (typically a higher, less deterministic temperature and a larger token ceiling). If your agent relied on the old deterministic defaults, set `temperature` and `max_tokens` explicitly. The upside: models that reject `max_tokens` (such as GPT-5.x) now work without workarounds — carry a limit on `max_completion_tokens` via `extra_params` where a model needs it.

Model key for local development: set `PORTKEY_API_KEY` in your `.env` to run against the gateway from your machine. In production the key comes from a secure secret instead (see Deployment). Full variable list in Reference.

Curated model presets (choose a model per request)

Instead of a single config, `model_config` can be a **map of named presets** you vet ahead of time and expose to callers (for example, a frontend model picker). The reserved `default` key is the base used when no preset is chosen:

```
model_config={
   "default":   ModelConfig(model_id="@<virtual-key>/eu.anthropic.claude-sonnet-4-6"),
   "quality":   ModelConfig(model_id="@<virtual-key>/global.anthropic.claude-opus-4-8",
max_tokens=8192),
   "reasoning": ModelConfig(model_id="@<virtual-key>/gpt-5.5", extra_params=
{"max_completion_tokens": 4096}),
}
```

A caller selects one for a **single request** with the **`x-aifoundry-model-preset`** header — no code change, no redeploy:

```
curl -X POST https://<your-agent>/invocations \
  -H "x-aifoundry-model-preset: reasoning" \
```

```
-H "Content-Type: application/json" \
-d '{ ... }'
```

- The header names a **configured preset only.** An unknown value is rejected up front — `400` on a chat (AG-UI) agent, or a JSON-RPC `InvalidParams` error on a backend (A2A) agent — with a message listing the valid presets. It is **not** a way to pass an arbitrary model slug.
- A bare `ModelConfig` (the single-config form above) still works and behaves as the implicit `default` preset. A preset map **without** a `default` key is a startup error.

Deployed agents must let the header through. The runtime only forwards headers listed in `request_header_allowlist`; newly scaffolded agents include `x-aifoundry-model-preset` by default. If it's missing, the header is stripped before it reaches your agent — the override works locally but silently no-ops once deployed:

```
"request_header_allowlist": ["Authorization", "x-aifoundry-model-preset"]
```

See Deployment → settings you control.

Raw model slugs (deprecated)

The older `x-portkey-model-slug` header still works for callers that need to pass an arbitrary gateway slug (`@<virtual-key>/<model-id>`) rather than a curated preset; it reconciles the request to universally-safe parameters and **logs a one-time deprecation warning per process**. Prefer curated presets. Don't send both headers with different values — the request is rejected.

Per-request Portkey options (advanced)

Callers can pass a small set of supported per-request **Portkey options** (routing metadata and gateway knobs) alongside a request — in A2A task metadata under `portkey_options`, or in an AG-UI request's `forwarded_props.portkey_options` (they ride in the request body, so no header allowlist entry is needed). You control whether this is allowed:

```
from ai_foundry_platform_sdk.agent import PortkeyConfig

agent = AgentDefinition(
  portkey=PortkeyConfig(
```

```
        allow_caller_kwargs=True,   # default — accept supported caller options
        debug_logging=False,        # default — see Observability for the debug switch
    ),
    ...
)
```

Credential, routing, and debug keys are **guarded** — a caller can never override them (see Security & Access → what you can't switch off).

Thinking / reasoning mode (advanced)

Some models support extended "thinking". To enable it, relax the gateway's strict compatibility and set a thinking budget:

```
# environment
PORTKEY_CLIENT_OPTIONS={"strict_open_ai_compliance": false}
```

```
model_config = ModelConfig(
    temperature=1.0,              # required for thinking mode
    max_tokens=16000,
    extra_params={"thinking": {"type": "enabled", "budget_tokens": 4096}},
)
```

Skills

A **skill** is a described capability of your agent. Every agent must declare at least one. Skills appear as metadata so other agents and tools can discover what your agent does — they don't contain logic themselves.

```
Skill(
    id="data-analysis",
    name="Data Analysis",
    description="Analyse CSV and Excel files, generate charts and summaries.",
    tags=["data", "analytics"],
)
```

Skill packages (optional, advanced)

Beyond simple skill metadata, you can attach **skill packages** — reusable capability bundles (a `SKILL.md` file plus optional helper files) that the model loads on demand. This keeps the agent's instructions lean: only a skill's name and description sit in context until the model decides to use it.

```
agent = AgentDefinition(
    ...,
    agent_root=Path(__file__).parent,
    skill_packages=[
        SkillPackage(source="local", location="skills/refunds"),     # on-disk bundle
        SkillPackage(source="url", location="https://example.com/SKILL.md"),  # hosted single file
    ],
)
```

`source="local"` points at a folder; `source="url"` at a single hosted `SKILL.md`. (Fetching from a git repo or an org registry is planned but not available yet.) Skill packages are validated once at startup, so a malformed package stops the agent from starting with a clear error.

Hooks (optional)

Hooks let you run your own logic at points in a request without subclassing anything. All are optional and may be synchronous or asynchronous.

```
from ai_foundry_platform_sdk.agent import AgentHooks
from ai_foundry_platform_sdk.agent.hooks import RequestContext

async def before_request(ctx: RequestContext) -> RequestContext:
    # runs after safety checks, before the agent sees the message
    return ctx

async def after_response(ctx: RequestContext, result):
    ...  # runs after the agent produces a result

agent = AgentDefinition(
    hooks=AgentHooks(before_request=before_request, after_response=after_response),
```

```
    ...
)
```

Available hook points: `before_request`, `after_response`, `on_error`, `on_tool_call`, and `extract_metadata` (customise how request metadata is read). The request context gives you the message parts, metadata, session ID, and the caller's identity (see Security & Access).

Multi-agent workflows

For agents that orchestrate other agents, use `build_agent(...)` to run a sub-agent in-process — it gets the same platform management (prompts, tools, cleanup) without its own server:

```
from ai_foundry_platform_sdk.app import build_agent
from ai_foundry_platform_sdk.model import ModelProvider

provider = ModelProvider()   # share one gateway client across sub-agents

with build_agent(analyst, model_provider=provider, session_id="abc") as a:
    analysis = a.agent("analyse the quarterly data")

with build_agent(summarizer, model_provider=provider, session_id="abc") as s:
    summary = s.agent(f"summarise: {analysis}")
```

Common patterns: a **sequential pipeline** (A's output feeds B), **parallel fan-out** (run several sub-agents at once), or a **router** (a parent agent picks which sub-agent to call).

Where next

- Tools & Files — give your agent real capabilities.
- Memory — remember conversations.
- Security & Access — control who can use it.
- Reference — every setting and environment variable.